

Hogwarts Arena

History: Remote Wizarding Education

*"It's 2025, and the magical world has been shaken up by an unexpected announcement. The Headmaster of Hogwarts, tired of the chaos caused by students running around casting spells, has decided that **remote education** is the way to go. No more wandering the corridors, no more wild broomstick races in the halls – everything will be managed online. In the middle of the Wizarding World's digital transformation, **the Headmaster has called upon you**, an aspiring wizarding web developer, to implement the **Wizarding Arena** platform. This web application will allow Hogwarts students to log in, learn spells virtually, track their magical power, and see how they stack up against their housemates.*

*Your task? To bring **Hogwarts Arena** to life! A platform where students can enter their accounts, hone their magical skills, and engage in some good old digital wizardry – all from the comfort of their own home.*

Are you ready to bring the magic to the web?"

1. Entities and Enums

There are **two main entities** in the system:

Wizard

This represents the user who is registered on the platform. Wizards will log in, learn spells, and track other wizards, edit their profile data and alignment.

- **id**: Unique identifier for each wizard (UUID)
- **username**: The wizard's username (String, **unique**, **not null**)
- **password**: The wizard's password (String, **not null**) - must be **hashed**
- **avatarUrl**: URL of the wizard's avatar (String, **not null**)
- **alignment**: The wizard's alignment (WizardAlignment, **not null**)
- **house**: The wizard's house (House, **not null**)
- **spells**: List of spells the wizard **has learned** (List<Spell>)
- **createdOn**: When the wizard registered (LocalDateTime, **not null**)
- **updatedOn**: When the wizard's information was last updated (LocalDateTime, **not null**)

Spell

This represents a spell that a wizard **has learned**. Each spell has various attributes:

- **id**: Unique identifier for each spell (UUID)
- **code**: The code of the spell (e.g., "LUMOS", "EXPELLIARMUS") (String, **not null**)

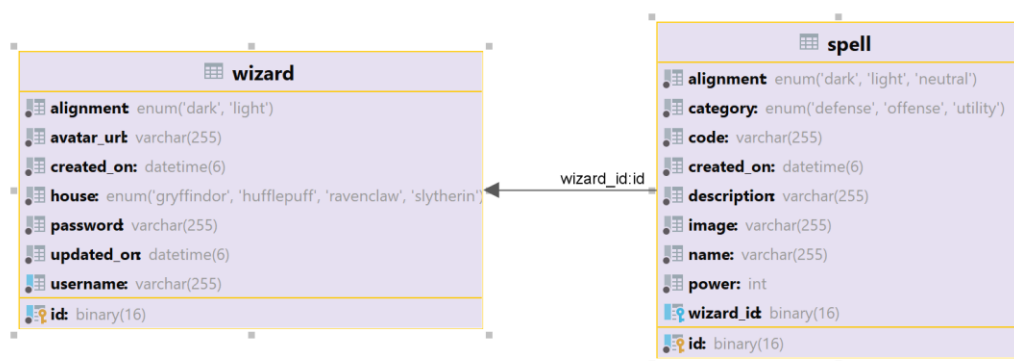
- **name:** The name of the spell (String, **not null**)
- **description:** A brief description of the spell (String, **not null**)
- **wizard:** The wizard who learned the spell (Wizard, **not null**)
- **category:** The category of the spell (SpellCategory, **not null**)
- **alignment:** The alignment of the spell (SpellAlignment, **not null**)
- **image:** The image of the spell (String, **not null**)
- **power:** The power of the spell (int)
- **createdOn:** Date when the spell was learned by the wizard (LocalDateTime, **not null**)

SpellCategory: OFFENSE, DEFENSE, UTILITY

SpellAlignment: LIGHT, NEUTRAL, DARK

WizardAlignment: LIGHT, DARK

House: GRYFFINDOR, SLYTHERIN, RAVENCLAW, HUFFLEPUFF



2. Configuration Properties

In the project skeleton, you are provided with a **spells.yaml** file containing the details of all available spells. This file will be used by the application to display spells to the wizard and manage which spells they can learn.

Each spell in the YAML contains the following fields:

- **code** - unique spell code (e.g. **LUMOS**)
- **name** - name of the spell
- **image** - spell's image (e.g. **/images/spells/lumos.png**)
- **category** - one of OFFENSE, DEFENSE, or UTILITY
- **alignment** - one of LIGHT, NEUTRAL, or DARK
- **minLearned** - how many spells a wizard must already know (have learned) before being allowed to learn this one
- **power** - numeric value representing the magical power for this spell
- **description** - description of the spell

Additionally, you are given an empty **SpellsProperties.java** class. Your task is to complete this class to read the **spells.yaml** file at application startup and map its content. The loaded data must be accessible across the application in runtime, allowing wizards to learn spells.

3. Business Constraints & Error Handling

You must **enforce every business rule** described in this document. If a user attempts an action that **violates a business constraint**, throw a **custom Java runtime exception** (e.g., only unique usernames, not learning the same spell twice, not learning a spell before meeting its *minLearned* requirement). The **WhiteLabel Error Page** is acceptable for the exam.

For pages with **form validation** - Register and Edit Profile forms, you must validate user input according to the listed constraints in the description. If invalid input is given upon form submit, you should **re-render the same form** and display the field error messages in the provided HTML. Do not use custom error messages, just use the message that is already provided in the HTML file.

4. Functionalities

Index Page

View: **index.html**

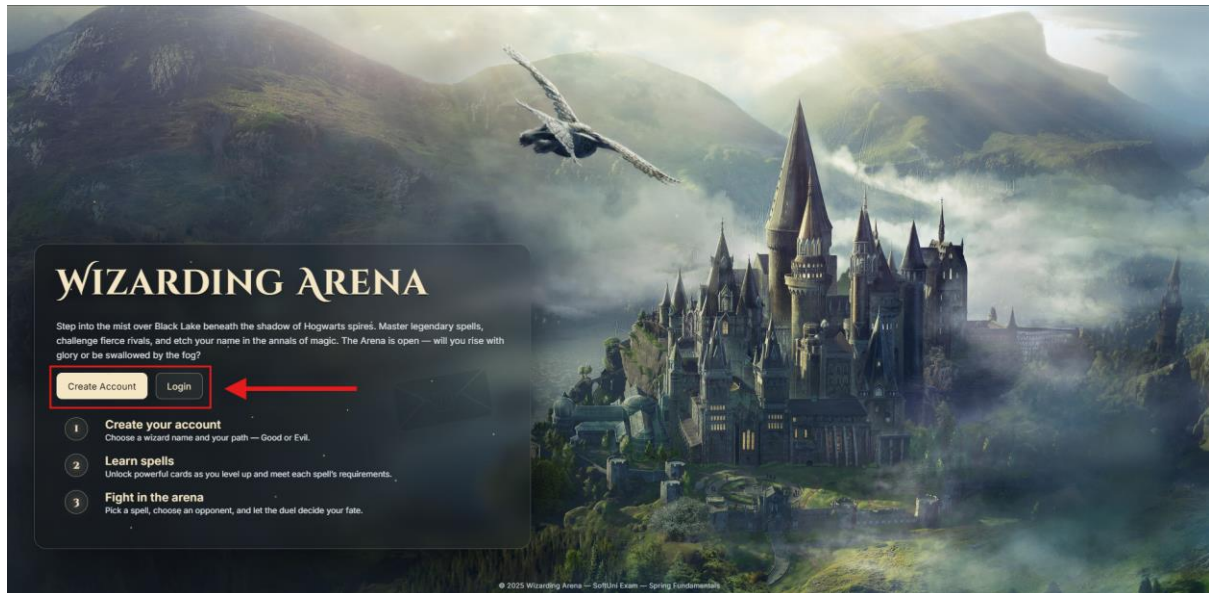
Mandatory endpoint: **"/"**

Requirements:

- Implement a **GET** handler that serves the **index.html view** as the application's landing page (*See Figure 1 below*).
- On this view, there are **two** main navigation links/buttons:
 - **Create Account** - navigates the user to the Register page
 - **Login** - navigates the user to the Login page



Figure 1: Default index view, landing page.



Register Page

View: **register.html**

Mandatory endpoint: **“/register”**

Requirements:

- Implement a **GET** handler that serves the **register.html** view (See Figure 1 below).
- Implement a **POST** handler that creates a new **Wizard** profile with the submitted form data.
 - Store the **password hashed**
 - Upon successful registration, the wizard must automatically learn 1 **random** spell from the spells list, but only from those that have **minLearned = 0**.
- Validate the form input, on **DTO field validation error**, show the same page with the error message provided in the HTML file - the small red messages, above the form input field (See Figure 2 below).
- You are given an **avatars.txt** file, where you can find existing avatar urls for your convenience to use, when you test your solution.
- Form validation constraints:
 - **Username** - length **[6 - 12]** characters, not empty/blank text
 - **Password** - exactly **6** characters, must include **only digits**, not empty/blank text
 - **Avatar URL** - must be a valid **URL**, not empty/blank text
 - **House** - must be valid **House enum value**, not null
 - **Alignment** - must be valid **WizardAlignment enum value**, not null
- On successful registration: navigates the user to the Login page.

Figure 1: Registration page, default view

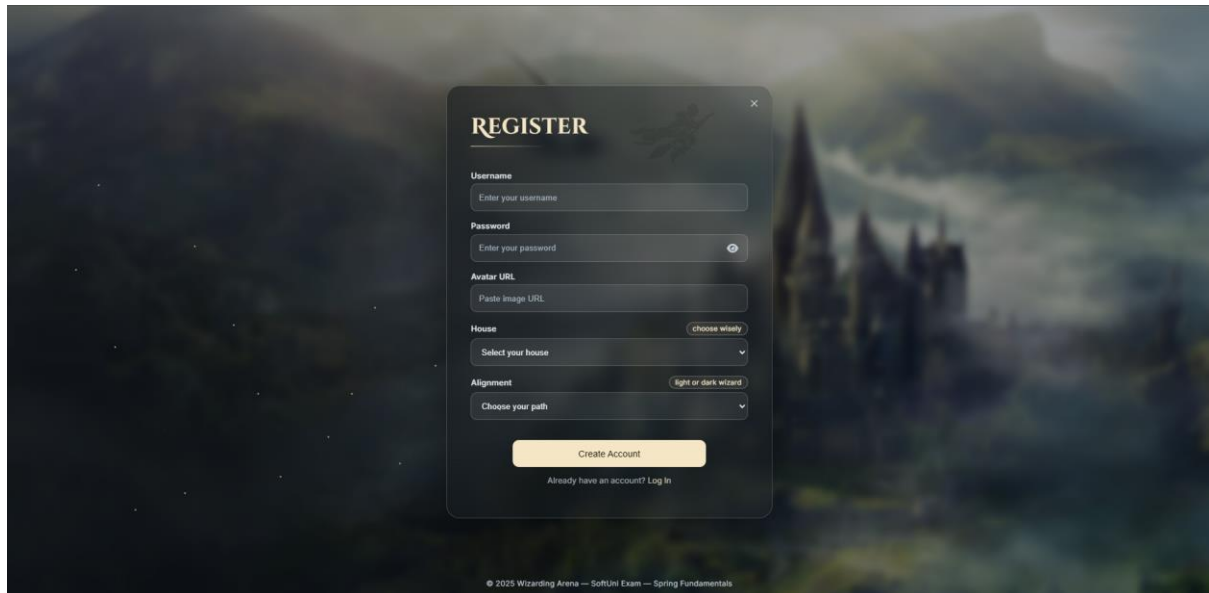
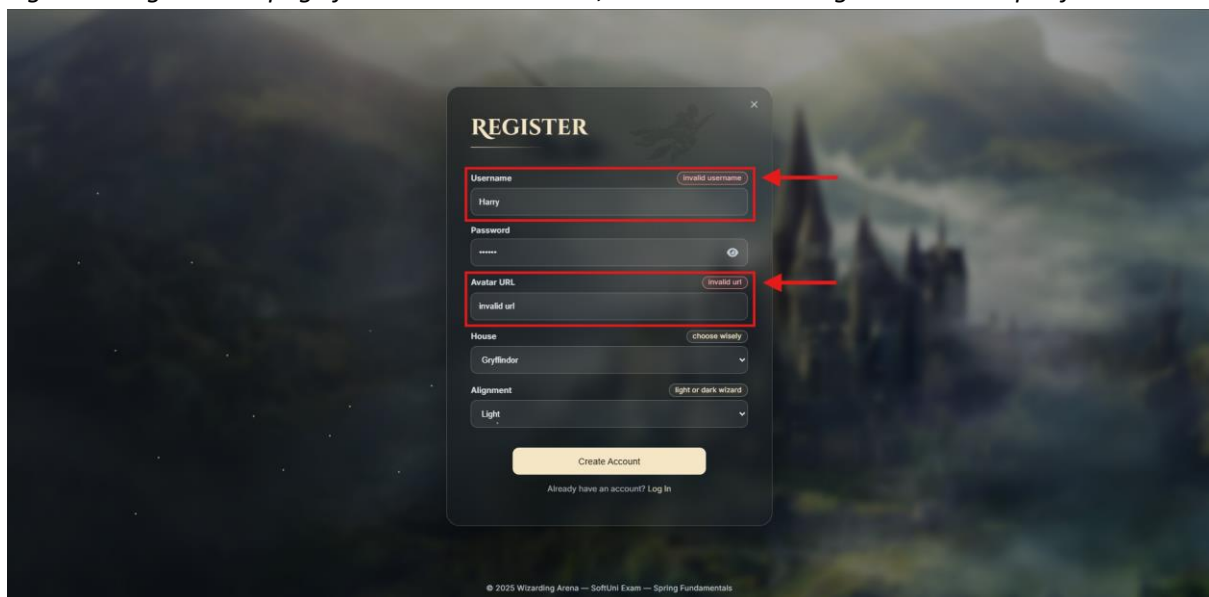
A screenshot of a web application's registration page. The page has a dark, atmospheric background featuring a castle on a hill. A central, semi-transparent modal window titled "REGISTER" contains the registration form. The form includes input fields for "Username" (placeholder: "Enter your username"), "Password" (placeholder: "Enter your password" with a toggle icon), "Avatar URL" (placeholder: "Paste image URL"), and dropdown menus for "House" (placeholder: "Select your house"), "Alignment" (placeholder: "Choose your path"), and "House" (placeholder: "choose wisely"). There are also buttons for "Create Account" and "Already have an account? Log In". At the bottom of the modal, a small copyright notice reads "© 2025 Wizarding Arena — SoftUni Exam — Spring Fundamentals".

Figure 2: Registration page filled with invalid data, shows error message above the input field

A screenshot of the same registration page as in Figure 1, but with invalid data entered. The "Username" field contains "Harry" and the "Avatar URL" field contains "invalid url". Red boxes highlight these two fields, and red arrows point to error messages: "invalid username" above the Username field and "invalid url" above the Avatar URL field. The "House" dropdown is set to "Gryffindor" and the "Alignment" dropdown is set to "Light". The "Create Account" button is visible at the bottom of the modal. The same copyright notice is at the bottom.

Login Page

View: **login.html**

Mandatory endpoint: **"/login"**

Requirements:

- Implement a **GET** handler that serves the **login.html** view (See Figure 1 below).
- Implement a **POST** handler that authenticates a wizard using **username** and **password**.
- On successful authentication: create a session and store the wizard's ID in a session attribute named **user_id**, then navigate the user to the Home page.
- On incorrect credentials (wizard not found or password mismatch): throw a **custom Java exception**.

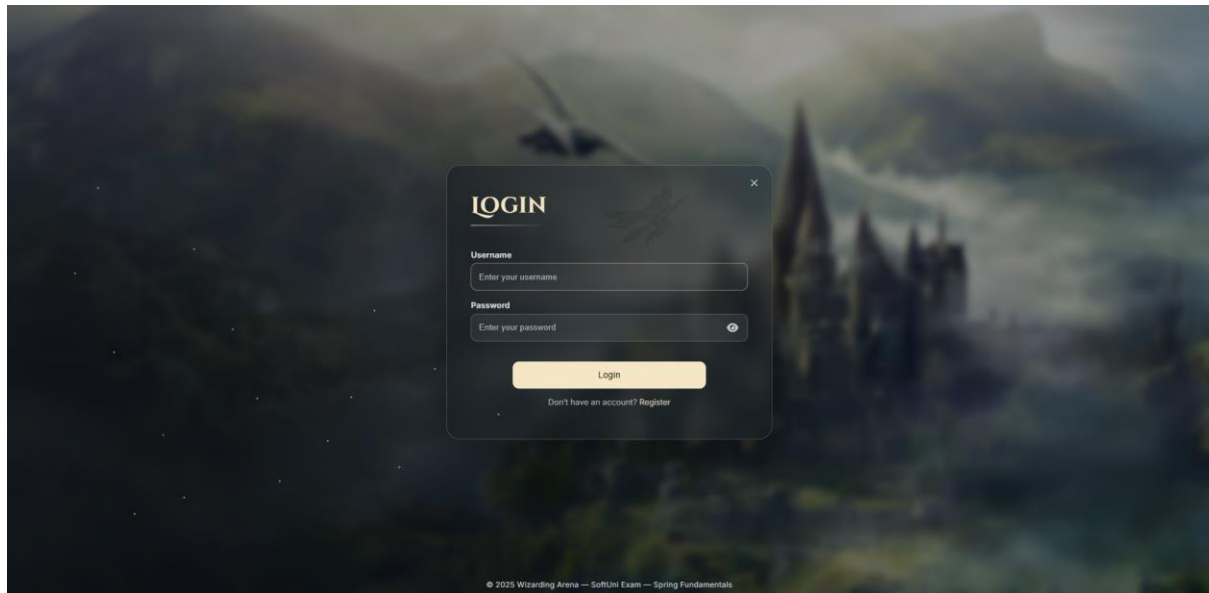


SoftUni

© SoftUni — about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Follow us: 

Figure 1: Login page, default view



Logout

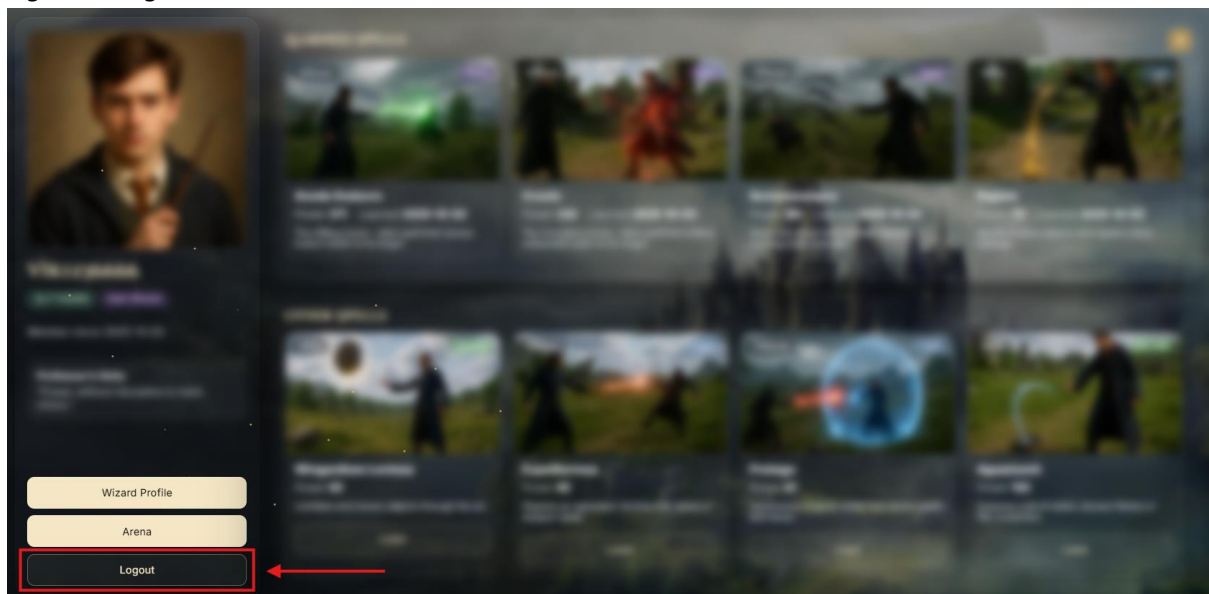
View: **Redirect to index page**

Mandatory endpoint: **“/logout”**

Requirements:

- Implement a **GET** handler that logs out the user and redirects them to the index page. The logout must happen upon clicking the “Logout” button in any page it exists. Upon successful logout, the **user’s session has to be invalidated!** (See Figure 1 below)

Figure 1: Logout button



Home Page



SoftUni

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Follow us:



View: **home.html**

Mandatory endpoint: **"/home"**

Requirements:

Implement a **GET** handler that serves the **home.html** view.

The home page contains **three main sections** you need to implement **carefully**:

- Wizard Profile Section
- Learned Spells Section
- Other Spells Section

Wizard Profile Section (See Figure 1 below)

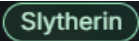
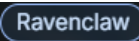
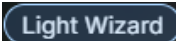
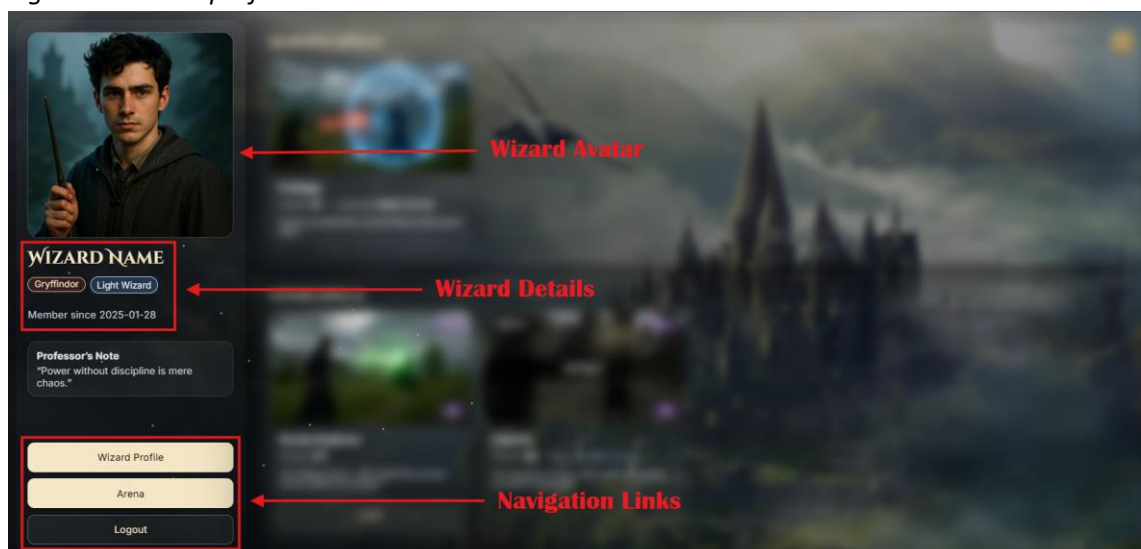
- Displays wizard details: avatar, username, house, alignment, "Member since {createdOn}"
- On this section, there are **three** main navigation links/buttons:
 - **Wizard Profile** - navigates the user to the Profile page.
 - **Arena** - navigates the user to the Arena page.
 - **Logout** - directly logs out the user, invalidates their session and redirects them to the Index page.
- The **house badge** reflects the wizard's house. Show one of these possible options:
 -    
- The **alignment badge** reflects the wizard's alignment. Show one of these possible options:
 -  

Figure 1: Wizard profile section



Learned Spells Section (See Figure 2 below)

- Header: *Learned Spells* with a **counter** for you to show how many spells the wizard has learned.
- Displays all spells the wizard has already learned and for each learned spell show - image, category, alignment, name, power, "Learned {createdOn}", description

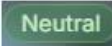
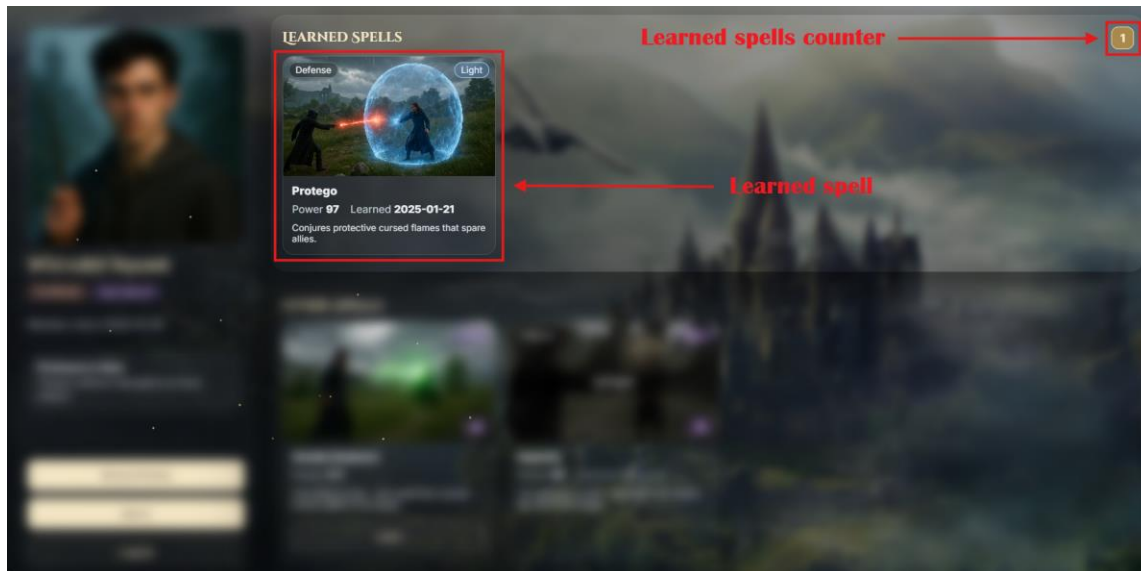
- The spells must be **ordered by power descending**.
- The **spell alignment batch** reflects the spell's alignment. Show one of these possible options:
 -   
- The **spell category badge** must be displayed with the **first letter capitalized only** (e.g., *Offense, Defense, Utility*). There are **no different styles** for that badge.

Figure 2: Learned spells section



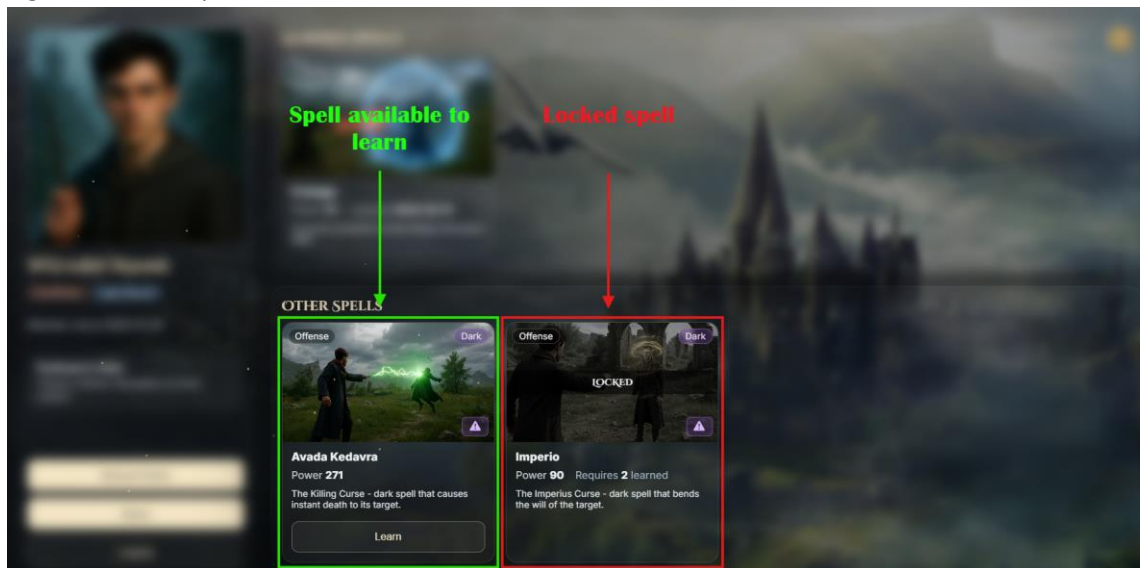
Other Spells Section (See Figure 3 below)

- Load all spells from the **spells.yaml** file that the wizard has **not learned yet**. Show their details - image, category, alignment, name, power, description, and warning badge (read below for this badge).
- Spells in this section are separated into two groups
 - **Available to Learn** - spells where the wizard meets the minLearned requirement. Each of these spells will be available for the wizard to learn.
 - **Locked** - spells where the wizard does not meet the minLearned requirement (they haven't learned the required number of spells yet).
- No specific ordering is required in this section.
- The **spell alignment batch** reflects the spell's alignment. Show one of these possible options:
 -   
- The **spell category badge** must be displayed with the **first letter capitalized only** (e.g., *Offense, Defense, Utility*). There are **no different styles** for this badge.
- If the logged-in user is **Light wizard** and the spell in the section is a **Dark spell**, display the **warning badge** , otherwise this badge must not be displayed!
- **Learn spell functionality** - when a wizard clicks on the "Learn" button a **POST** handler must be implemented to handle the user's request (e.g. /spells?spell-code=LUMOS). A new learned **Spell** must be created for the wizard based on the selected spell from **spells.yaml**. Do not allow learning the same spell twice, already learned spells should ideally not appear in this section anymore. A wizard **cannot learn a spell** if the **minLearned** requirement is not

satisfied! **Every wizard can learn any spell**, regardless of the spell/wizard alignment, but **if a Light wizard** learns a **Dark spell**, the wizard's alignment instantly changes to **Dark**, while if a Dark wizard learns a Light spell, there are no consequences (their alignment stays Dark). After a spell successfully learned, redirect the user to the Home page again, they should be able to see the newly learned spell in the Learned Spells Section.

- About **minLearned** - every spell in the **spells.yaml** file has a field called minLearned. This number tells how many spells a wizard must have already learned before they can be able to learn this spell.

Figure 3: Other spells section



Profile Page

View: **profile.html**

Requirements:

Implement a **GET** handler that serves the **profile.html** view.

On the page there is a **Back to Home** button that you should make to navigate the user to the **Home page**.

The page also shows two panels you need to implement:

Edit Profile (See Figure 1 below)

- Implement a **PUT** handler that allows the wizard to edit their username and avatar url.
- On page load, the form fields **must be pre-filled** with the wizard's current data (username and avatar URL)
 - Hint: Instead of passing an empty object DTO for the form (as you normally would), set the DTO fields with the wizard's details and then pass it.
- Validate the form input, on **DTO field validation error**, show the same page with the error message provided in the HTML file - the small red messages, above the form input field.
- Form validation constraints:

- **Username** - length [6 - 12] characters, not empty/blank text
- **Avatar URL** - must be a valid **URL**, not empty/blank text
- On submit: update the user's data with the incoming username and avatar, set updatedOn to now and persist the changes.

Alignment Change (See Figure 2 below)

- Implement a **PATCH** handler that changes the wizard's alignment **from Light to Dark**.
- If the wizard is already **Dark**: don't show a button/form element, but display the static paragraph provided in the HTML document - "You have chosen the Dark path. This choice is permanent."

Figure 1: Profile page, edit profile panel

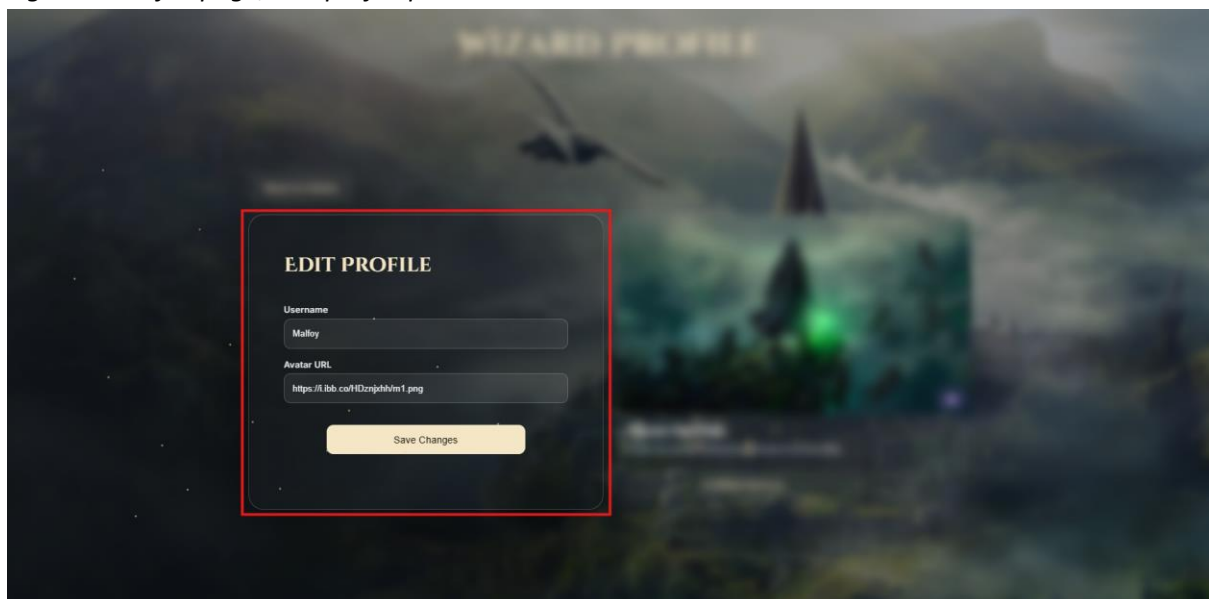
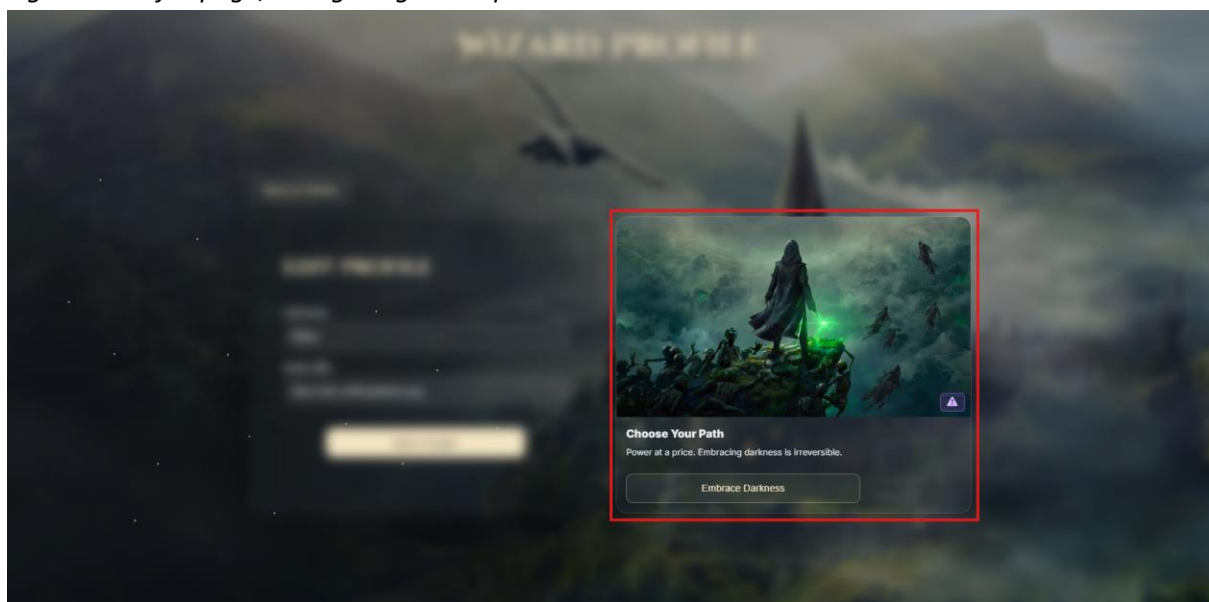


Figure 2: Profile page, change alignment panel



5. Security

The security is based on an active session. Upon login, **you must** store the user's unique identifier (id) in a session attribute named **user_id**.

A **SessionInterceptor** class is provided in the project skeleton - **do not modify it**.

Access control **already enforced** as part of the **SessionInterceptor** class:

Public: Index Page, Register Page, Login Page, Logout

Restricted (only logged-in users): Home, Arena, Profile, and all related functionalities

6. Project Upload

When uploading your solution, you must create an archive containing the **src** folder and the **pom.xml** file.

Go to the **Regular Exam** topic, click 'ИЗБОР НА ФАЙЛ', select the archived file, and click **Upload/КАЧИ**.

Please note that **only your last uploaded file will be saved**.

Important Note: Projects will be tested for functionality. Any project that does not run successfully will receive **0 points**.

7. Optional functionality

Important Note: The **Arena page** is **optional**. You can achieve the full score for this exam without implementing this page. However, if implemented, it may serve as **compensation** for missing parts in other sections!

Arena Page

View: **arena.html**

Requirements:

- Implement a **GET** handler that serves the **arena.html** view (*See Figure 1 below*).
- The page shows **four fixed rows**, one per house: Gryffindor, Slytherin, Ravenclaw, Hufflepuff
- Each house row lists the wizards **belonging to that house** as horizontal cards. For each wizard card show:
 - Avatar, username, alignment badge.
 - Spells **N** (number of learned spells)
 - Power **Σ** (sum of the wizard's learned spells' power)
- The **alignment badge** reflects the wizard's alignment. Show one of these possible options:
 - 
- Each house must display the **count of wizards** in that house.

- **Sorting inside each house row:** by **total power descending**, then by **username ascending**.
(See Figure 1 below for an example)
- If a house has **no wizards**, show the empty state **“No students yet.”**, that is provided in the HTML document and **count of wizards 0** (See Figure 2 below)
- There is a **Back to Home** button placed **above the Gryffindor row (top-left corner)** that you should make to navigate the user to the **Home** page.

Figure 1: Arena page, default view

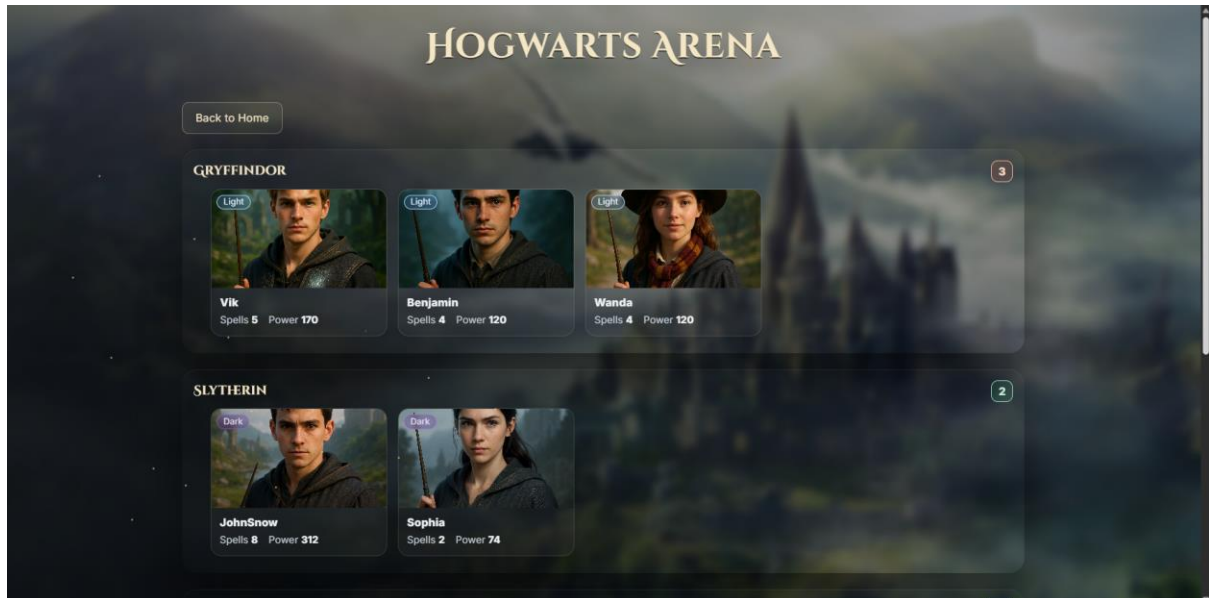


Figure 2: Arena page with a house with no students

